Lecture 9: Abstract classes and inheritance

Lifting common fields and methods, Inheritance

Related files:
IShape.java AShapeData.java AShape.java
AShapeCombo.java

Video Lessons

- Lesson 1 - Abstract Classes and Inheritance

Outline

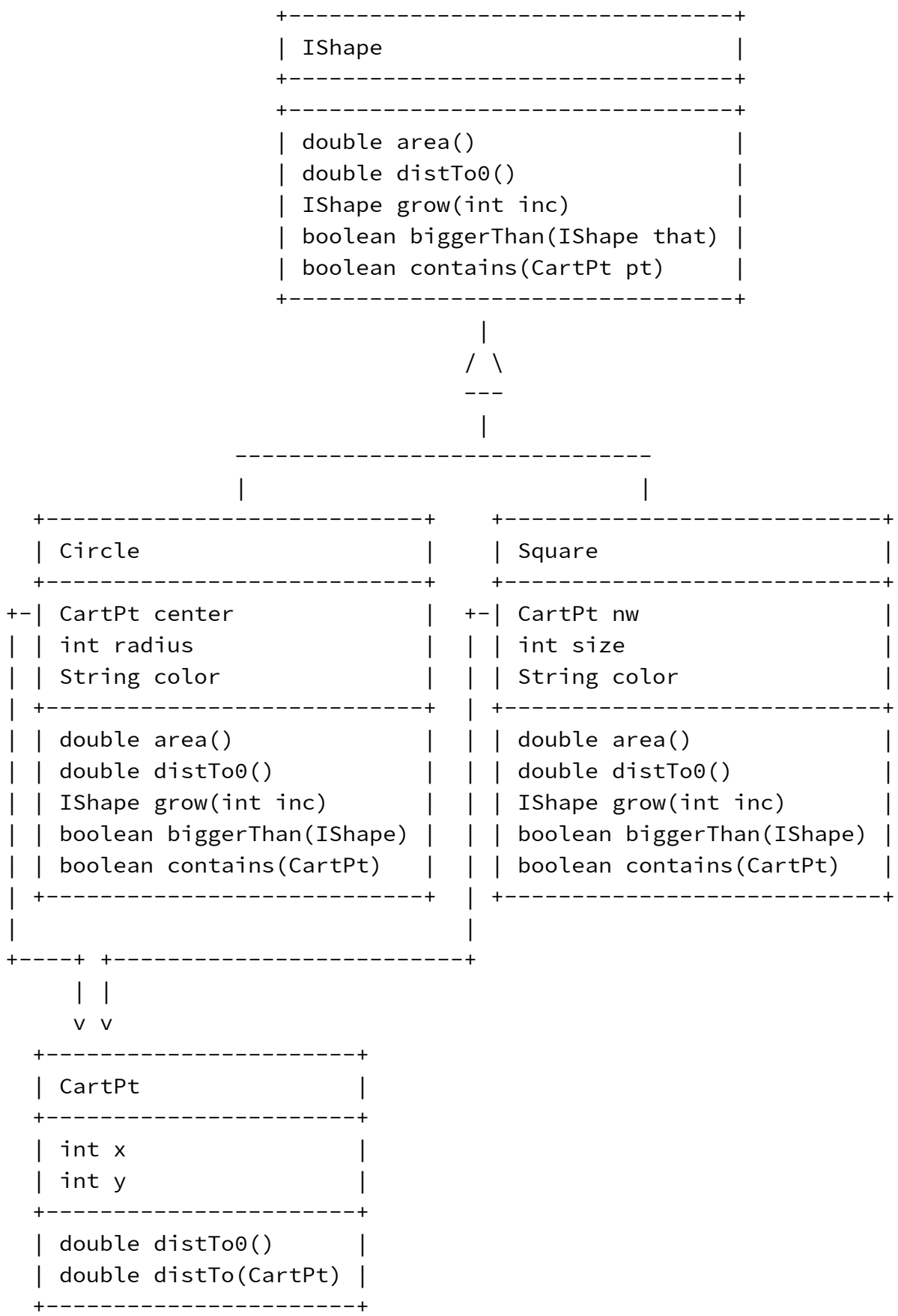
- Design recipe for abstractions
- Designing abstract class
  - Lifting fields
  - Lifting methods: abstract methods
  - Lifting methods: concrete methods in an abstract class
  - Overriding concrete methods
- Abstraction by defining a subclass
- Common interface - yes or no

9.1 Design recipe for abstractions

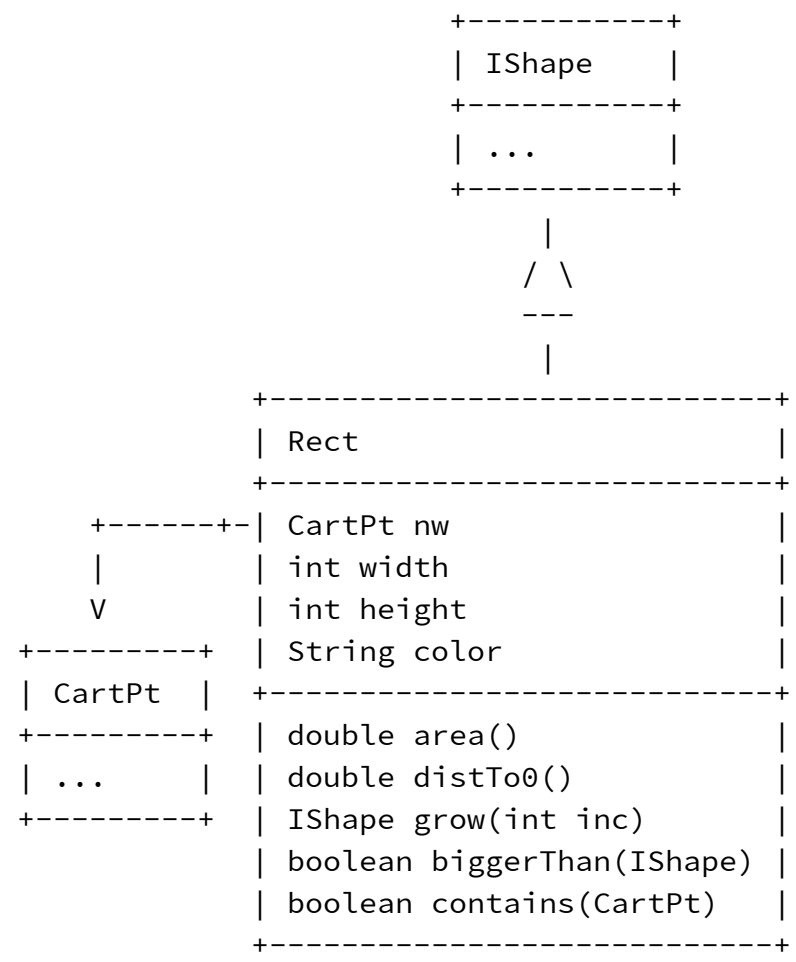
The complete code for this section is in the file `IShape.java`.

Here are our two classes that represent shapes that we have defined to *implement* the common `interface IShape`.

The class diagram below includes the five methods we have designed for these classes.



Our task now is to extend this class hierarchy with a new class, to represent rectangles.



Since rectangles are rather like squares, it's worthwhile to review the definition of the `Square` class for initial design guidance:

```
// to represent a square
class Square implements IShape {
  CartPt nw;
  int size;
  String color;

  Square(CartPt nw, int size, String color) {
    this.nw = nw;
    this.size = size;
    this.color = color;
  }

  /* TEMPLATE
    FIELDS
    ... this.nw ...                -- CartPt
    ... this.size ...              -- int
    ... this.color ...             -- String
    METHODS
    ... this.area() ...                    -- double
    ... this.distTo0() ...                 -- double
```

```
    ... this.grow(int inc) ...            -- IShape
    ... this.biggerThan(IShape that) ... -- boolean
    ... this.contains(CartPt pt) ...      -- boolean
    METHODS FOR FIELDS:
    ... this.nw.distTo0() ...         -- double
    ... this.nw.distTo(CartPt) ...    -- double
  */

  // to compute the area of this shape
  public double area() {
    return this.size * this.size;
  }

  // to compute the distance form this shape to the origin
  public double distTo0() {
    return this.nw.distTo0();
  }

  // to increase the size of this shape by the given increment
  public IShape grow(int inc) {
    return new Square(this.nw, this.size + inc, this.color);
  }

  // is the area of this shape is bigger than the area of the given shape?
  public boolean biggerThan(IShape that) {
    return this.area() >= that.area();
  }

  // does this shape (including the boundary) contain the given point?
  public boolean contains(CartPt pt) {
    return (this.nw.x <= pt.x) && (pt.x <= this.nw.x + this.size) &&
           (this.nw.y <= pt.y) && (pt.y <= this.nw.y + this.size);
  }
}
```

Generalizing from this definition to an appropriate definition of `Rect` is straightforward. We will skip the explicit details of the design recipe here, and just give the final definition of the new class:

```
// to represent a rectangle
class Rect implements IShape {
  CartPt nw;
  int width;
  int height;
  String color;
```

```
Rect(CartPt nw, int width, int height, String color) {
  this.nw = nw;
  this.width = width;
  this.height = height;
  this.color = color;
}

/* TEMPLATE
  FIELDS
  ... this.nw ...              -- CartPt
  ... this.width ...           -- int
  ... this.height ...          -- int
  ... this.color ...           -- String
  METHODS
  ... this.area() ...                  -- double
  ... this.distTo0() ...               -- double
  ... this.grow(int inc) ...           -- IShape
  ... this.biggerThan(IShape that) ... -- boolean
  ... this.contains(CartPt pt) ...     -- boolean
  METHODS FOR FIELDS:
  ... this.nw.distTo0() ...        -- double
  ... this.nw.distTo(CartPt) ...   -- double
*/

// to compute the area of this shape
public double area() {
  return this.width * this.height;
}

// to compute the distance form this shape to the origin
public double distTo0() {
  return this.nw.distTo0();
}

// to increase the size of this shape by the given increment
public IShape grow(int inc) {
  return new Rect(this.nw, this.width + inc, this.height + inc,
                  this.color);
}

// is the area of this shape is bigger than the area of the given shape?
public boolean biggerThan(IShape that) {
  return this.area() >= that.area();
}
```

```
  // does this shape (including the boundary) contain the given point?
  public boolean contains(CartPt pt) {
    return (this.nw.x <= pt.x) && (pt.x <= this.nw.x + this.width) &&
           (this.nw.y <= pt.y) && (pt.y <= this.nw.y + this.height);
  }
}
```

We do need to add examples, to ensure our methods are working properly:

```
IShape r1 = new Rect(new CartPt(50, 50), 30, 20, "red");
IShape r2 = new Rect(new CartPt(50, 50), 50, 40, "red");
IShape r3 = new Rect(new CartPt(20, 40), 10, 20, "green");

  // test the method area in the class Rect
boolean testRectArea(Tester t) {
  return
  t.checkInexact(this.r1.area(), 600.0, 0.01);
}

// test the method distTo0 in the class Rect
boolean testRectDistTo0(Tester t) {
  return
  t.checkInexact(this.r1.distTo0(), 70.71, 0.01) &&
  t.checkInexact(this.r3.distTo0(), 44.72, 0.01);
}

// test the method grow in the class Rect
boolean testRectGrow(Tester t) {
  return
  t.checkExpect(this.r1.grow(20), this.r2);
}

// test the method biggerThan in the class Rect
boolean testRectBiggerThan(Tester t) {
  return
  t.checkExpect(this.r1.biggerThan(this.r2), false) &&
  t.checkExpect(this.r2.biggerThan(this.r1), true) &&
  t.checkExpect(this.r1.biggerThan(this.r1), true) &&
  t.checkExpect(this.r3.biggerThan(this.r1), false);
}

// test the method contains in the class Rect
boolean testRectContains(Tester t) {
```

```
    return
    t.checkExpect(this.r1.contains(new CartPt(100, 100)), false) &&
    t.checkExpect(this.r2.contains(new CartPt(55, 60)), true);
  }
```

Notice that a lot of code is repeated. Moreover, every shape includes a location (a `CartPt` value) and a color (a `String`) as part of its definition. Such repetition should suggest that perhaps there is a way to *abstract* the common features of the code, to avoid these duplicate definitions. Recall the **Design Recipe for Abstractions**:

- Compare two or more pieces of code that look very similar.

  Highlight the places where they differ.

- Replace the places where the code differs with parameters and rewrite the original code using their parameters.

- Rewrite the original tests in terms of the new *abstraction* by providing the appropriate values for the arguments.

- Now run the original tests and make sure all tests pass.

9.2 Lifting fields

The complete code for this section is in the file `AShapeData.java`

We will start by working on the data definitions only, ignoring the methods defined in these classes.

All three classes contain a field of type `String` representing the shape's color, and also a field of type `CartPt` representing the shape's location. Note that while the color field is named `color` in all three classes, the location field does not have the same name in all three classes.

> ***Do Now!***
>
> The location field is named `center` in the `Circle` class and represents the location of the center of the circle, but is named `nw` in the `Square` and `Rect` classes, and represents the location of the north-west corner of the square or rectangle. Does it make sense to consider these to be "the same" field? Why or why not?

Java allows us to define an `abstract` class to contain these common fields, and declare that each of our three shape classes `extends` this class. As a naming convention, our abstract classes will have names starting with a capital 'A', much as our interfaces always start with a capital 'I'.

```
// to represent a geometric shape
abstract class AShape implements IShape {
  CartPt loc;
  String color;

  AShape(CartPt loc, String color) {
    this.loc = loc;
    this.color = color;
  }
}
```

Just like class definitions we have seen already, this class defines a constructor to initialize its fields, and claims that it `implements IShape`

> But we have not implemented any of the methods that the interface promises, and yet Java does not complain! We'll see why shortly.

Do Now!

Does it make sense to be able to construct an `AShape`? What would that mean?

We cannot plausibly construct an *abstract* shape—we don't have enough information about it—and a *real* shape will have other fields to represent that additional information.

Each of the three classes will become a *subclass* of the class `AShape` and will *inherit* all fields defined in its *super* class:

```
// to represent a circle
class Circle extends AShape {
  int radius;

  Circle(CartPt center, int radius, String color) {
    super(center, color);
    this.radius = radius;
```

```
  }
}

// to represent a square
class Square extends AShape {
  int size;

  Square(CartPt nw, int size, String color) {
    super(nw, color);
    this.size = size;
  }
}

// to represent a rectangle
class Rect extends AShape {
  int width;
  int height;

  Rect(CartPt nw, int width, int height, String color) {
    super(nw, color);
    this.width = width;
    this.height = height;
  }
}
```

Even though we do not see the fields `loc` and `color` anywhere in the class definitions, the class definition starts with

```
class Circle extends AShape { ... }

class Square extends AShape { ... }

class Rect extends AShape { ... }
```

declaring that this class will be a sub-class of the class `AShape` and so it contains all fields defined in the super class.

The constructor for each class starts with

`super(nw, color)` (though we use `ctr` for the `Circle` class).

It invokes the constructor we have defined in the class `AShape` and initializes the values of the fields `loc` and `color`.

We have re-named the field that represents the current location of the shape to `loc`,

but that does not change any computations or examples. Indeed, we make sure that the original examples will work as they did before.

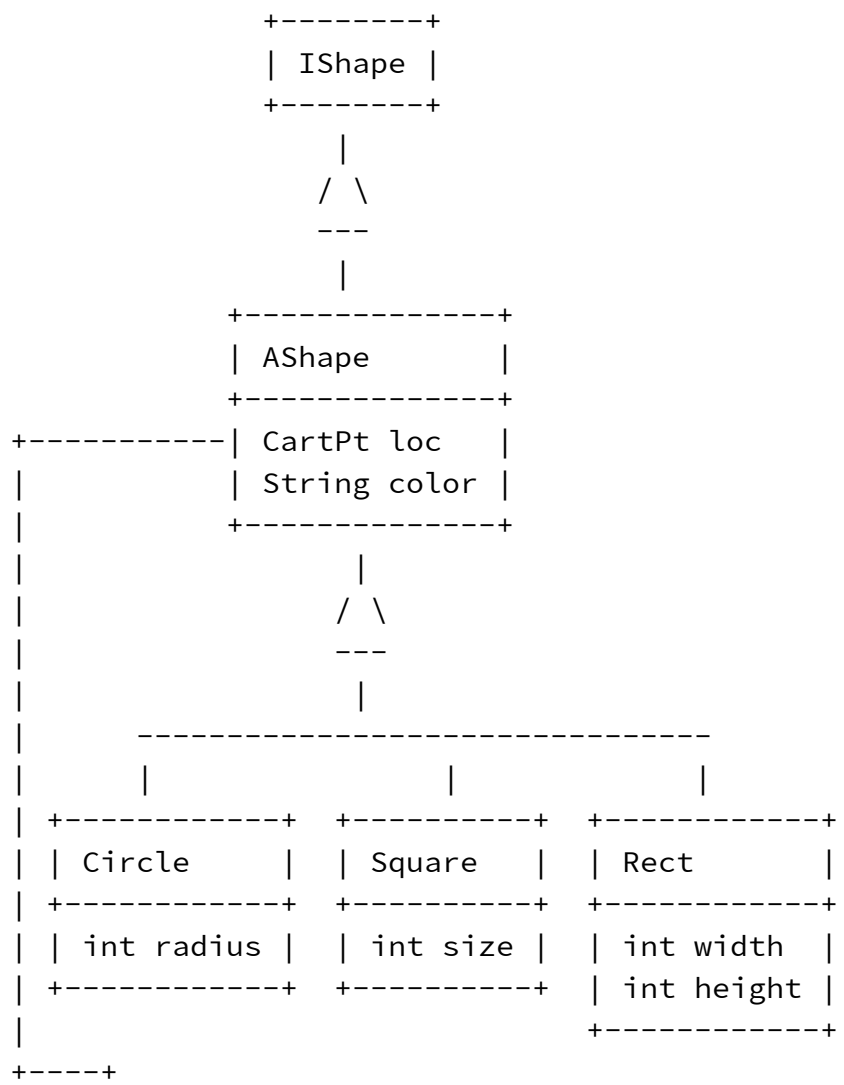
```
CartPt pt1 = new CartPt(0, 0);
CartPt pt2 = new CartPt(3, 4);
CartPt pt3 = new CartPt(7, 1);

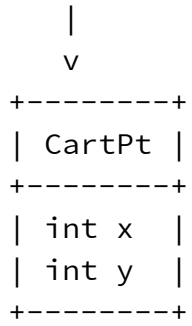
IShape c1 = new Circle(new CartPt(50, 50), 10, "red");
IShape c2 = new Circle(new CartPt(50, 50), 30, "red");
IShape c3 = new Circle(new CartPt(30, 100), 30, "blue");

IShape s1 = new Square(new CartPt(50, 50), 30, "red");
IShape s2 = new Square(new CartPt(50, 50), 50, "red");
IShape s3 = new Square(new CartPt(20, 40), 10, "green");

IShape r1 = new Rect(new CartPt(50, 50), 30, 20, "red");
IShape r2 = new Rect(new CartPt(50, 50), 50, 40, "red");
IShape r3 = new Rect(new CartPt(20, 40), 10, 20, "green");
```

Our new class diagram becomes:





9.3 Lifting methods: abstract methods

The complete code for this section and the following section is in the file `AShape.java`

We now look at what can be done with the method definitions.

The `abstract` class does not have enough information about its constituent shapes to define the methods `area`, `grow`, `distTo0` (though we could do this for two out of the three classes), and `contains`. But the method `biggerThan` leverages the fact that we already know how to compute the area and the method body is the same in all three classes.

When we cannot define the methods in the abstract class, but want to make sure all *subclasses* implement the method, we define its header and declare the method to be `abstract`. So, to comply with the requirements of our original interface we start the abstract class as follows:

```
// to represent a geometric shape
abstract class AShape implements IShape {
  CartPt loc;
  String color;

  AShape(CartPt loc, String color) {
    this.loc = loc;
    this.color = color;
  }

  // to compute the area of this shape
  public abstract double area();

  // to compute the distance form this shape to the origin
  public double distTo0() {
    return this.loc.distTo0();
  }
```

```
  // to increase the size of this shape by the given increment
  public abstract IShape grow(int inc);

  // is the area of this shape is bigger than the area of the given shape?
  public boolean biggerThan(IShape that) {
    return this.area() >= that.area();
  }

  // does this shape (including the boundary) contain the given point?
  public abstract boolean contains(CartPt pt);
}
```

The definitions of the methods that are labeled `abstract` remains the same in all subclasses as they have been before (except for the change of the name of the field that represents the current location of this shape).

9.4 Concrete methods in the abstract class

The class definition contains two *concrete* methods: `distTo0` and `biggerThan`. The second method's body was the same in all three classes and so now it appears here without change. Every subclass of the class `AShape` can invoke this method.

We have made a small change in the method body for `distTo0` — we have changed the name of the field `nw` to `loc`. Again, all three subclasses can now compute the distance to origin invoking this method.

If we delete the method definitions for these methods from our original code and run our tests most of them will succeed. The only one that fails is the test for `distTo0` in the class `Circle`, because there the computation has been different. Restoring the original method definition in the class `Circle` fixes the problem.

We say that the method definition of the method `distTo0` in the class `Circle` **overrides** the definition in its superclass `AShape`. At runtime, Java looks for a method with the matching signature (matching header) first in the class where the current instance of the object has been defined. If the method is not found, it continues looking in its superclass. So, an instance of a `Circle` invoking the `distTo0` method finds the definition in the class `Circle`, but an instance of the `Rect` class will find the *concrete* method defined in the abstract class `AShape`.

Of course, we run the tests again and make sure all of them pass.

9.5 Abstraction by defining a subclass

Looking at the code for the classes `Square` and `Rect` we see that they are also very similar. We know from geometry that every square is just a rectangle in which the width and the height are the same. So, we can further refine our design and define the class `Square` to be a subclass of the class `Rect`. Changing our design in this way means that `Square` will now have `width` and `height` fields, and we do not want to permit these values to be different (or else it would be a badly-misshapen square!). We can ensure these values are the same by customizing our constructor. We'll see in Lecture 10: Customizing constructors for correctness and convenience more sorts of validation we can enforce in our constructors.

Here is our new definition of the class `Square`:

```
// to represent a square
class Square extends Rect {

  Square(CartPt nw, int size, String color) {
    super(nw, size, size, color);
  }

  /*  TEMPLATE
   Fields:
   ... this.loc ...              -- CartPt
   ... this.width ...            -- int
   ... this.height ...           -- int
   ... this.color ...            -- String
   Methods:
   ... this.area() ...                    -- double
   ... this.distTo0() ...                 -- double
   ... this.grow(int) ...                 -- IShape
   ... this.biggerThan(IShape) ...        -- boolean
   ... this.contains(CartPt) ...          -- boolean
   Methods for fields:
   ... this.loc.distTo0() ...             -- double
   ... this.loc.distTo(CartPt) ...        -- double
   */

  // to increase the size of this shape by the given increment
  public IShape grow(int inc) {
      return new Square(this.loc, this.width + inc, this.color);
  }
}
```

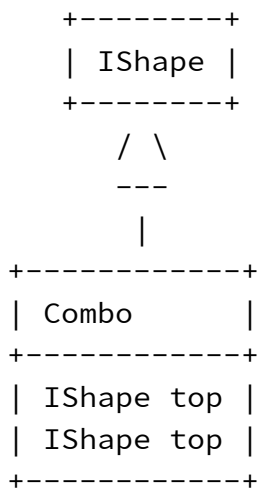
The constructor invokes the constructor in its superclass, `Rect`.

Furthermore, we only have one method definition here - the methods `distTo0` and `contains` as defined in the `Rect` class do exactly what they are supposed to do for squares - the only method we need to *override* is the method `grow` as it needs to produce an instance of a `Square`, not of `Rect`.

9.6 Common interface - yes or no

The complete code for this section is in the file AShapeCombo.java

It seems that defining both the interface `IShape` and the abstract class `AShape` just repeats the code and is *useless*. For the classes we have defined so far, it is indeed true. However, we would like to add to our collection of shapes a new shape that is composed from two existing shapes. The class diagram would be:



Here are some examples of `Combo` shapes:

```
IShape cb1 = new Combo(this.r1, this.c1);
IShape cb2 = new Combo(this.r2, this.r3);
IShape cb3 = new Combo(this.cb1, this.cb2);
```

leveraging the earlier examples of shapes.

But the `Combo` shape does not have just one color, and its location is based on the location of the two contributing shapes. Still, we can compute the area of this shape (though, to simplify our work, we just compute the total area of the pieces of paper if one makes a collage by pasting the shapes onto a canvas). We can compute the distance to origin, and determine which shape has bigger area. Finally, we also can grow this shape.

So, all methods defined earlier make sense for the `Combo` shape, yet the `Combo` shape cannot be a subclass of `AShape`. But if both the class `AShape` and the class `Combo`

implement the common interface `IShape`, Java will be able to invoke these five methods for any kind of shape, whether simple or a complex one.

Here is the definition of the class `Combo` — we let you work out the details on your own:

```
// to represent a shape that combines two existing shapes
class Combo implements IShape {
  IShape top;
  IShape bot;

  Combo(IShape top, IShape bot) {
    this.top = top;
    this.bot = bot;
  }

  /* TEMPLATE
  FIELDS
  ... this.top ...           -- IShape
  ... this.bot ...           -- IShape
  METHODS
  ... this.area() ...                  -- double
  ... this.distTo0() ...               -- double
  ... this.grow(int) ...               -- IShape
  ... this.biggerThan(IShape) ...      -- boolean
  ... this.contains(CartPt) ...        -- boolean
  METHODS FOR FIELDS:
  ... this.top.area() ...                  -- double
  ... this.top.distTo0() ...               -- double
  ... this.top.grow(int) ...               -- IShape
  ... this.top.biggerThan(IShape) ...      -- boolean
  ... this.top.contains(CartPt) ...        -- boolean

  ... this.bot.area() ...                  -- double
  ... this.bot.distTo0() ...               -- double
  ... this.bot.grow(int) ...               -- IShape
  ... this.bot.biggerThan(IShape) ...      -- boolean
  ... this.bot.contains(CartPt) ...        -- boolean
  */
  // to compute the area of this shape
  public double area() {
    return this.top.area() + this.bot.area();
  }

  // to compute the distance form this shape to the origin
```

```
  public double distTo0() {
    return Math.min(this.top.distTo0(), this.bot.distTo0());
  }

  // to increase the size of this shape by the given increment
  public IShape grow(int inc) {
    return new Combo(this.top.grow(inc), this.bot.grow(inc));
  }

  // is the area of this shape is bigger than the area of the given shape?
  public boolean biggerThan(IShape that) {
    return this.area() >= that.area();
  }

  // does this shape (including the boundary) contain the given point?
  public boolean contains(CartPt pt) {
    return this.top.contains(pt) || this.bot.contains(pt);
  }
}
```

and here are the examples that involve the `Combo` shapes:

```
// test the method area in the shape classes
boolean testShapeArea(Tester t) {
    return
    t.checkInexact(this.cb1.area(), 914.15926, 0.01) &&
    t.checkInexact(this.cb2.area(), 2200.0, 0.01) &&
    t.checkInexact(this.cb3.area(), 3114.15926, 0.01);
}

// test the method distTo0 in the shape classes
boolean testShapeDistTo0(Tester t) {
    return
    t.checkInexact(this.cb1.distTo0(), 60.71, 0.01) &&
    t.checkInexact(this.cb2.distTo0(), 44.72, 0.01) &&
    t.checkInexact(this.cb3.distTo0(), 44.72, 0.01);
}

// test the method grow in the shape classes
boolean testShapeGrow(Tester t) {
    return
    t.checkExpect(this.cb1.grow(20), new Combo(this.r2, this.c2));
}

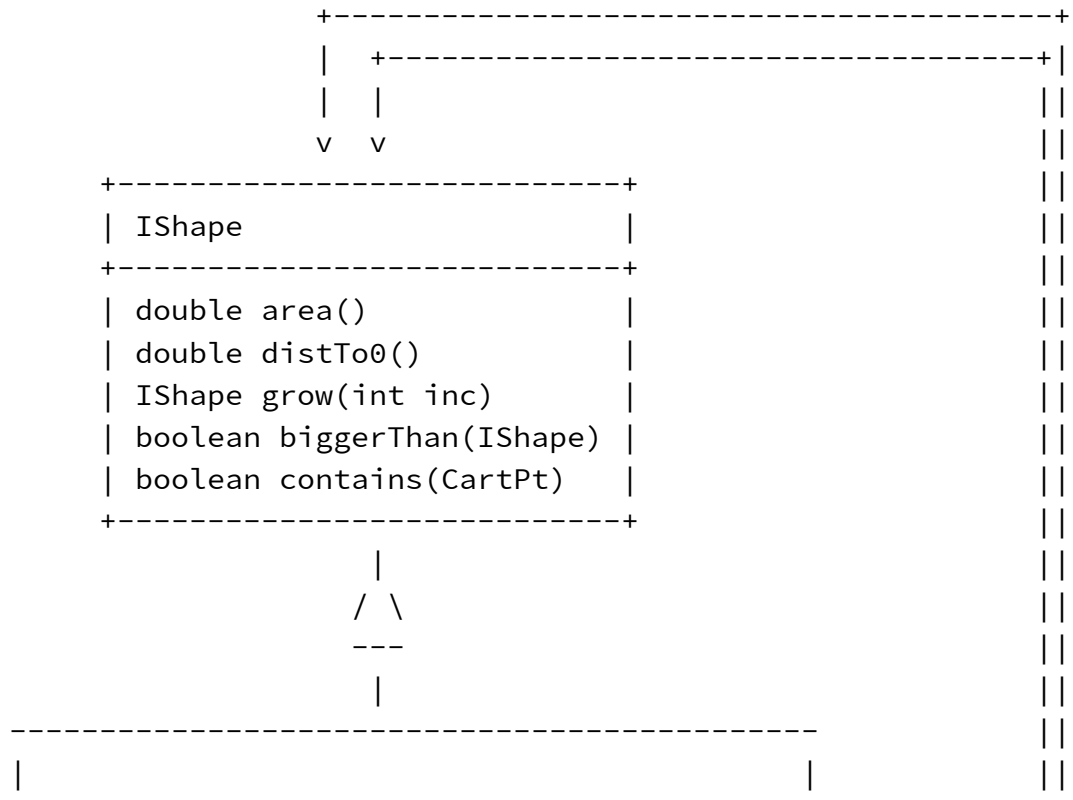
// test the method biggerThan in the shape classes
```

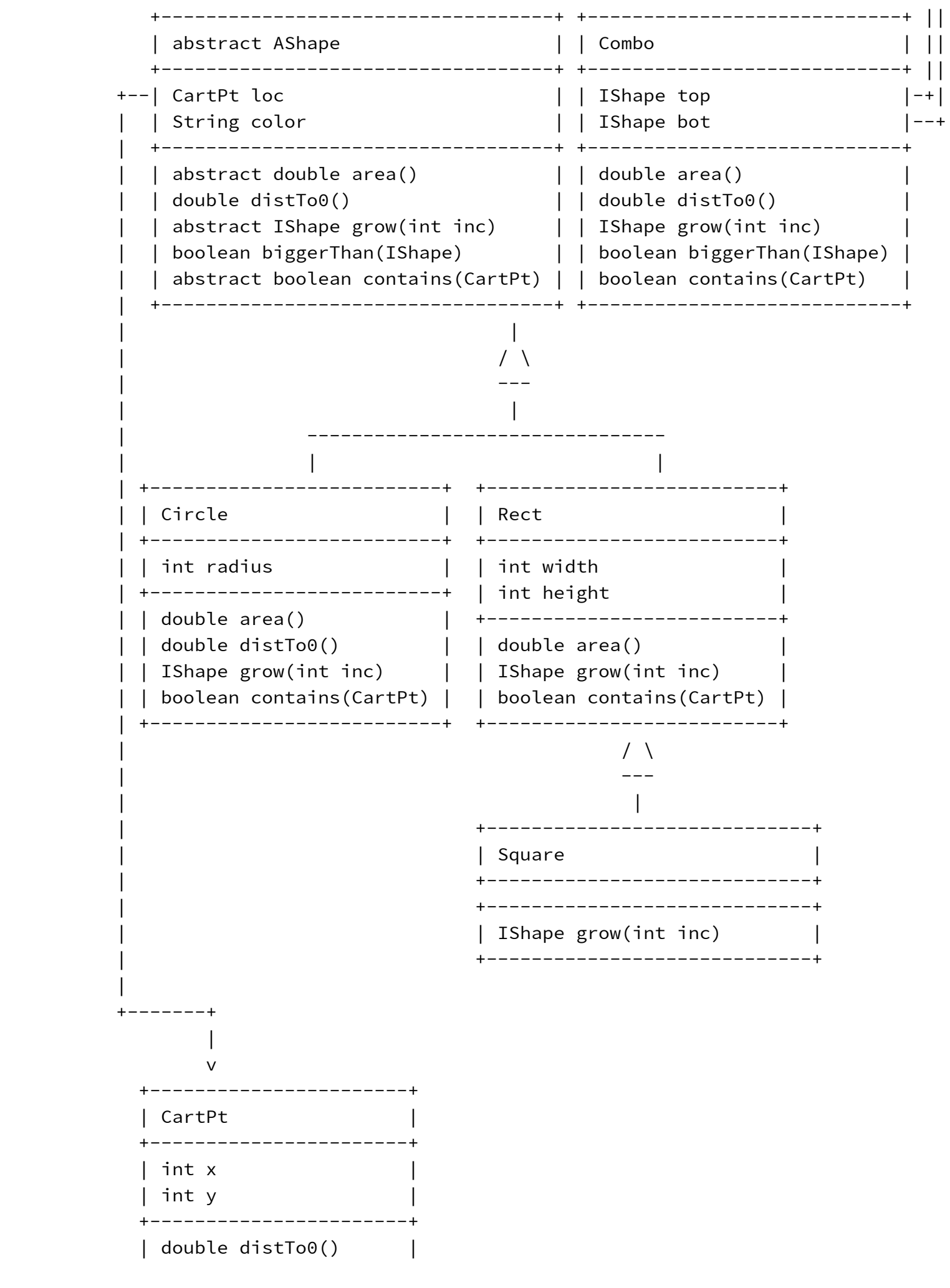
```
boolean testShapeBiggerThan(Tester t) {
    return
    t.checkExpect(this.c1.biggerThan(this.cb1), false) &&
    t.checkExpect(this.s2.biggerThan(this.cb1), true) &&
    t.checkExpect(this.r2.biggerThan(this.cb1), true) &&
    t.checkExpect(this.r3.biggerThan(this.cb1), false) &&

    t.checkExpect(this.cb2.biggerThan(this.r1), true) &&
    t.checkExpect(this.cb1.biggerThan(this.r2), false) &&
    t.checkExpect(this.cb1.biggerThan(this.c1), true) &&
    t.checkExpect(this.cb1.biggerThan(this.c3), false) &&
    t.checkExpect(this.cb1.biggerThan(this.s2), false) &&
    t.checkExpect(this.cb2.biggerThan(this.s1), true) &&
    t.checkExpect(this.cb1.biggerThan(this.cb3), false) &&
    t.checkExpect(this.cb2.biggerThan(this.cb1), true);
}

// test the method contains in the shape classes
boolean testShapeContains(Tester t) {
    return
    t.checkExpect(this.cb1.contains(new CartPt(100, 100)), false) &&
    t.checkExpect(this.cb2.contains(new CartPt(55, 60)), true);
}
```

Finally, here is the class diagram for the entire collection of classes and interfaces we have designed:




abstract AShape
CartPt loc
String color
abstract double area()
double distTo0()
abstract IShape grow(int inc)
boolean biggerThan(IShape)
abstract boolean contains(CartPt)
Combo
IShape top
IShape bot
double area()
double distTo0()
IShape grow(int inc)
boolean biggerThan(IShape)
boolean contains(CartPt)
Circle
int radius
double area()
double distTo0()
IShape grow(int inc)
boolean contains(CartPt)
Rect
int width
int height
double area()
IShape grow(int inc)
boolean contains(CartPt)
Square
IShape grow(int inc)
CartPt
int x
int y
double distTo0()

```
| double distTo(CartPt) |
+-----------------------+
```